COMP 412
FALL 2012

Context-sensitive Analysis
or
Semantic Elaboration

Comp 412

Copyright 2012, Keith D. Cooper & Linda Torczon, all rights reserved.
Students enrolled in Comp 412 at Rice University have explicit permission to make copies of these materials for their personal use.
Faculty from other educational institutions may use these materials for nonprofit educational purposes, provided this copyright notice is preserved.

Beyond Syntax

There is a level of correctness that is deeper than grammar

```
fie(a,b,c,d) {
    int a, b, c, d;
     …
}
fee() {
    int f[3],g[0], h, i, j, k;
    char *p;

    fie(h,i,“ab”,j, k);
    k = f * i + j;
    h = g[17];
    printf(“<%s,%s>.\n”,p,q);
    p = 10;
}
```

What is wrong with this program?

(let me count the ways …)

- number of args to fie()
- declared g[0], used g[17]
- “ab” is not an <u>int</u>
- wrong dimension on use of f
- undeclared variable q
- 10 is not a character string

All of these are

“deeper than syntax”

To generate code, we need to understand its meaning !

Beyond Syntax

To generate code, the compiler needs to answer many questions

- Is "x" a scalar, an array, or a function? Is "x" declared?
- Are there names that are not declared? Declared but not used?
- Which declaration of "x" does a given use reference?
- Is the expression "x * y + z" type-consistent?
- In "*a*[i,j,k]", does *a* have three dimensions?
- Where can "z" be stored? *(register, local, global, heap, static)*
- In "f ← 15", how should 15 be represented?
- How many arguments does "fie()" take? What about "printf ()" ?
- Does "*p" reference the result of a "malloc()" ?
- Do "p" & "q" refer to the same memory location?
- Is "x" defined before it is used?

These are beyond the expressive power of a CFG

Beyond Syntax

These questions are part of context-sensitive analysis

- Answers depend on values, not parts of speech
- Questions & answers involve non-local information
- Answers may involve computation

How can we answer these questions?

- Use formal methods
  - Context-sensitive grammars?
  - Attribute grammars? *(attributed grammars?)*
- Use *ad-hoc* techniques
  - Symbol tables
  - *Ad-hoc* code *(action routines)*

In parsing, formalisms won.

In context-sensitive analysis, ad-hoc techniques dominate practice.

Beyond Syntax

Telling the story

- We will study the formalism — an attribute grammar
  - Clarify many issues in a succinct and immediate way
  - Separate analysis problems from their implementations
- We will see that the problems with attribute grammars motivate actual, *ad-hoc* practice
  - Non-local computation
  - Need for centralized information
- Some folks still argue for attribute grammars
  - Knowledge is power
  - Information is immunization

We will cover attribute grammars, then move on to *ad-hoc* ideas

Attribute Grammars

What is an attribute grammar?

- A context-free grammar augmented with a set of rules
- Each symbol in the derivation (or parse tree) has a set of named values, or *attributes*
- The rules specify how to compute a value for each attribute
  - Attribution rules are functional; they uniquely define the value

Example grammar

1	*Number*	→	*Sign List*
2	*Sign*	→	+
3		|	-
4	*List*	→	*List Bit*
5		|	*Bit*
6	*Bit*	→	0
7		|	1

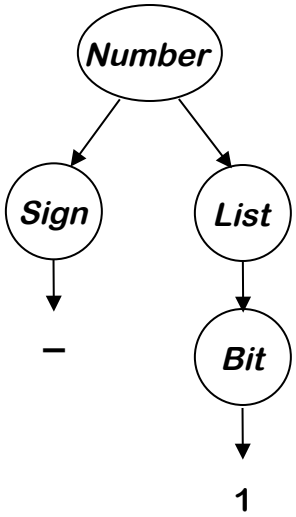
This grammar describes signed binary numbers

We would like to augment it with rules that compute the decimal value of each valid input string

Examples

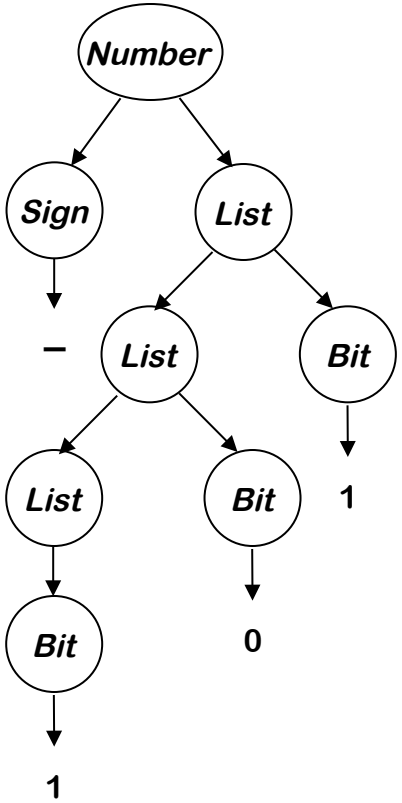
For "–1"

Number → *Sign List*
→ *Sign Bit*
→ *Sign 1*
→ *– 1*



For "–101"

Number → *Sign List*
→ *Sign List Bit*
→ *Sign List 1*
→ *Sign List Bit 1*
→ *Sign List 0 1*
→ *Sign Bit 0 1*
→ *Sign 1 0 1*
→ *– 101*



We will use these two examples throughout the lecture

Attribute Grammars

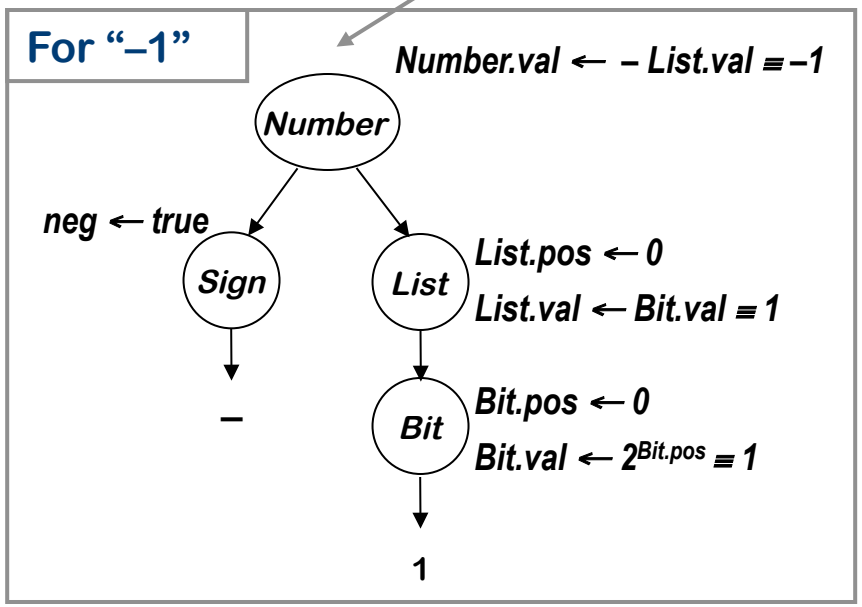
Add rules to compute the decimal value of a signed binary number

Productions			Attribution Rules
Number	→	*Sign List*	*List.pos* ← 0 *if Sign.neg* *then Number.val* ← - *List.val* *else Number.val* ← *List.val*
Sign	→	+	*Sign.neg* ← *false*
	|	-	*Sign.neg* ← *true*
$List_0$	→	$List_1$ *Bit*	$List_1.pos \leftarrow List_0.pos + 1$ $Bit.pos \leftarrow List_0.pos$ $List_0.val \leftarrow List_1.val + Bit.val$
	|	*Bit*	*Bit.pos* ← *List.pos* *List.val* ← *Bit.val*
Bit	→	0	*Bit.val* ← 0
	|	1	$Bit.val \leftarrow 2^{Bit.pos}$

Symbol	Attributes
Number	val
Sign	neg
List	pos, val
Bit	pos, val

Back to the Examples

Rules + parse tree imply an attribute dependence graph



One possible evaluation order:

1 List.pos
2 Sign.neg
3 Bit.pos
4 Bit.val
5 List.val
6 Number.val

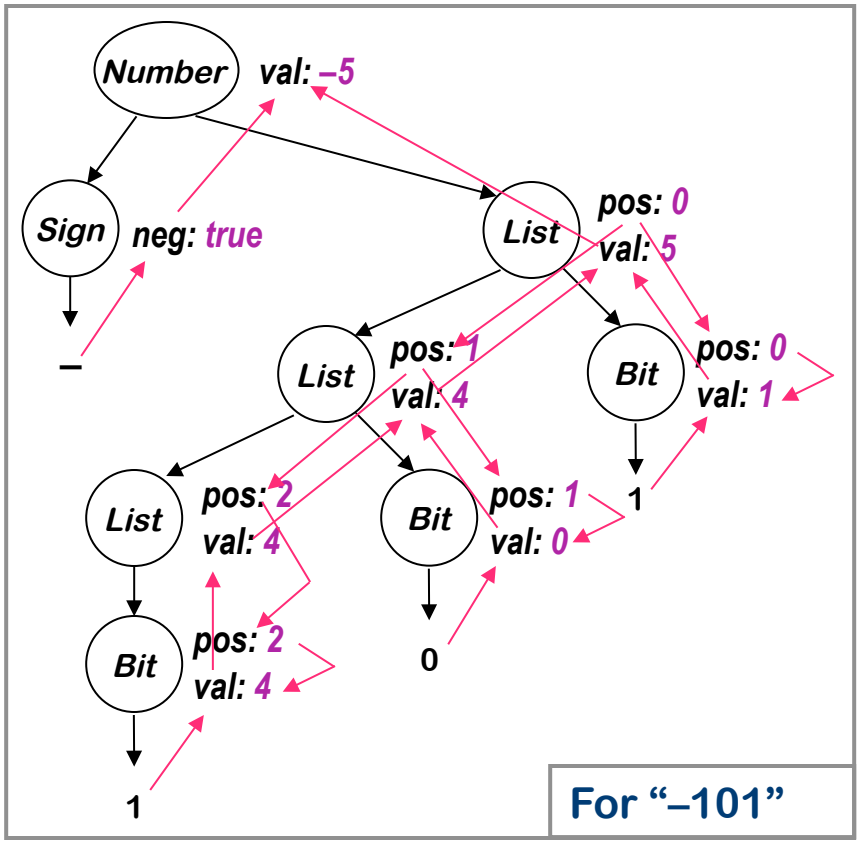
Other orders are possible

Knuth suggested a data-flow model for evaluation

- Independent attributes first
- Others in order as input values become available

Evaluation order must be consistent with the attribute dependence graph

Back to the Examples



This is the complete attribute dependence graph for "–101".

It shows the flow of *all* attribute values in the example.

Some flow *downward*

→ inherited attributes

Some flow *upward*

→ synthesized attributes

A rule may use attributes in the parent, children, or siblings of a node

The Rules of the Game

- Attributes associated with nodes in parse tree
- Rules are value assignments associated with productions
- Attribute is defined once, using local information
- Label identical terms in production for uniqueness
- Rules & parse tree define an attribute dependence graph
  - Graph must be non-circular

This produces a high-level, functional specification

N.B.: AG is a specification for the computation, not an algorithm

Synthesized attribute

- Depends on values from children

Inherited attribute

- Depends on values from siblings & parent

Using Attribute Grammars

Attribute grammars can specify context-sensitive actions

- Take values from syntax
- Perform computations with values
- Insert tests, logic, …

Synthesized Attributes

- Use values from children & from constants
- S-attributed grammars
- Evaluate in a single bottom-up pass

Good match to LR parsing

Inherited Attributes

- Use values from parent, constants, & siblings
- Directly express context
- Can rewrite to avoid them
- Thought to be more *natural*

Not easily done at parse time

We want to use both kinds of attributes

Evaluation Methods

Dynamic, dependence-based methods

- Build the parse tree
- Build the dependence graph
- Topological sort the dependence graph
- Define attributes in topological order

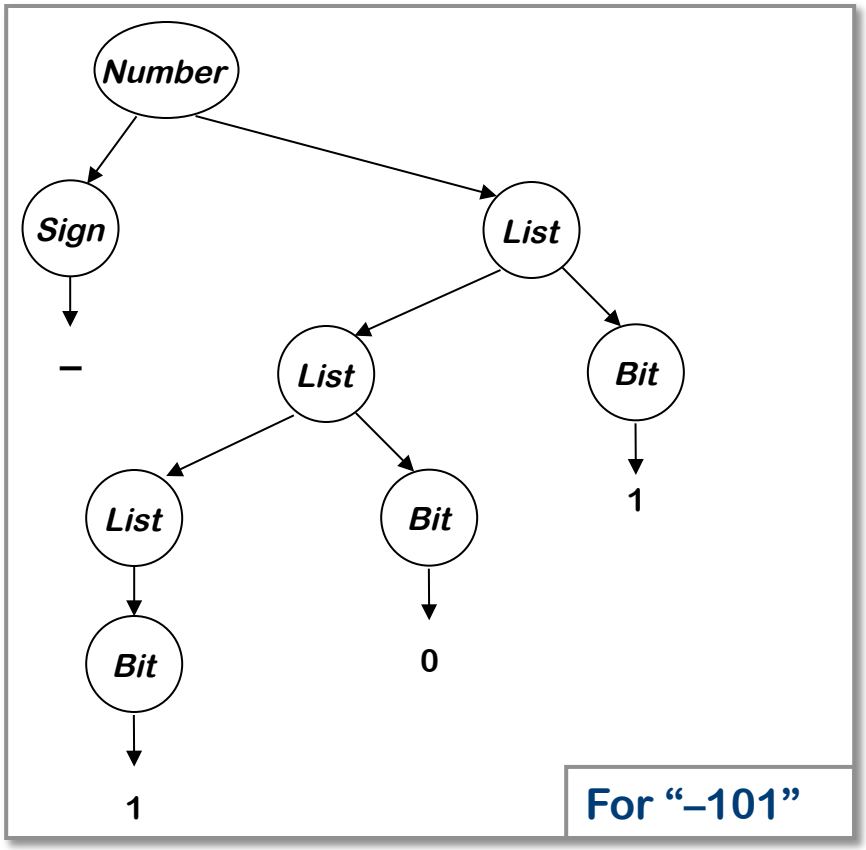
Rule-based methods *(treewalk)*

- Analyze rules at compiler-generation time
- Determine a fixed (static) ordering
- Evaluate nodes in that order

Oblivious methods *(passes, dataflow)*

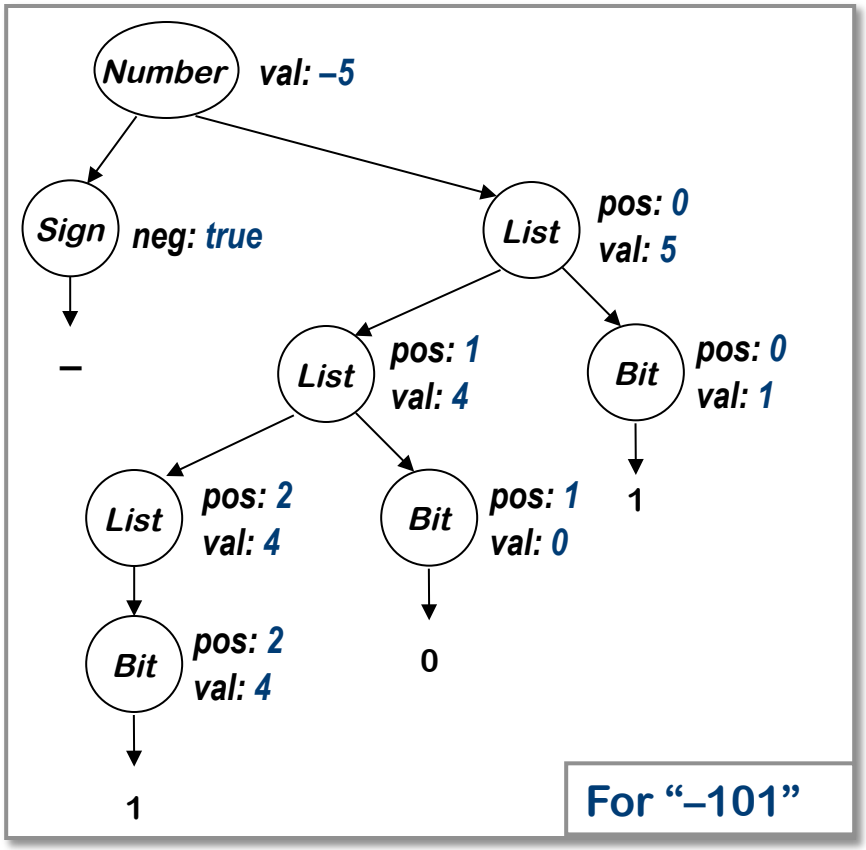
- Ignore rules & parse tree
- Pick a convenient order (at design time) & use it

Back to the Example



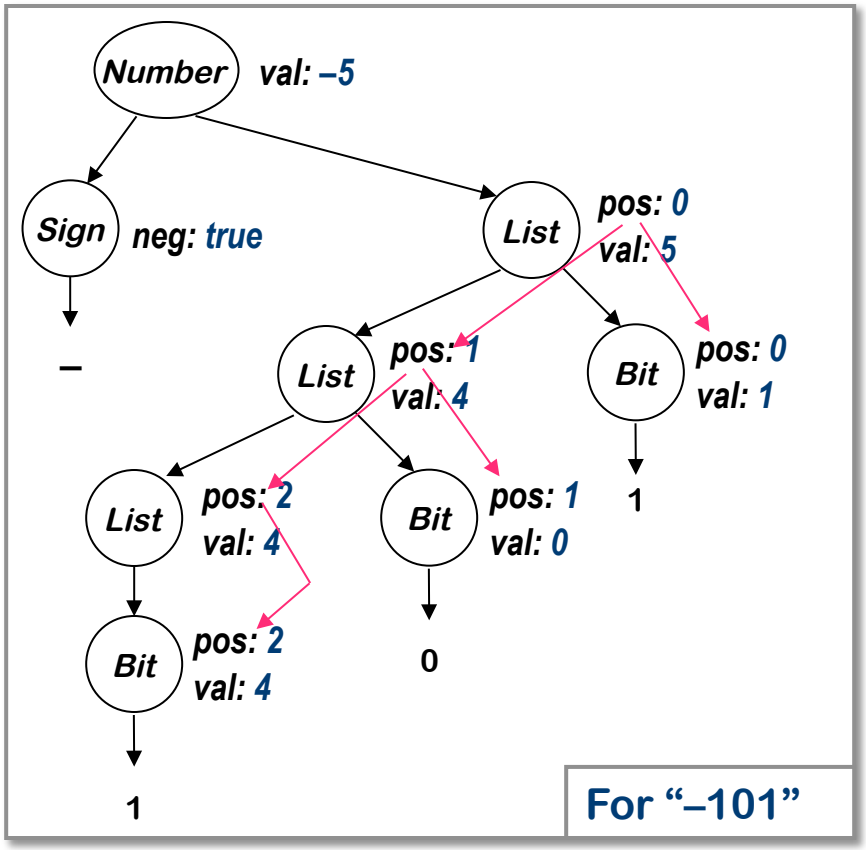
Syntax Tree

Back to the Example



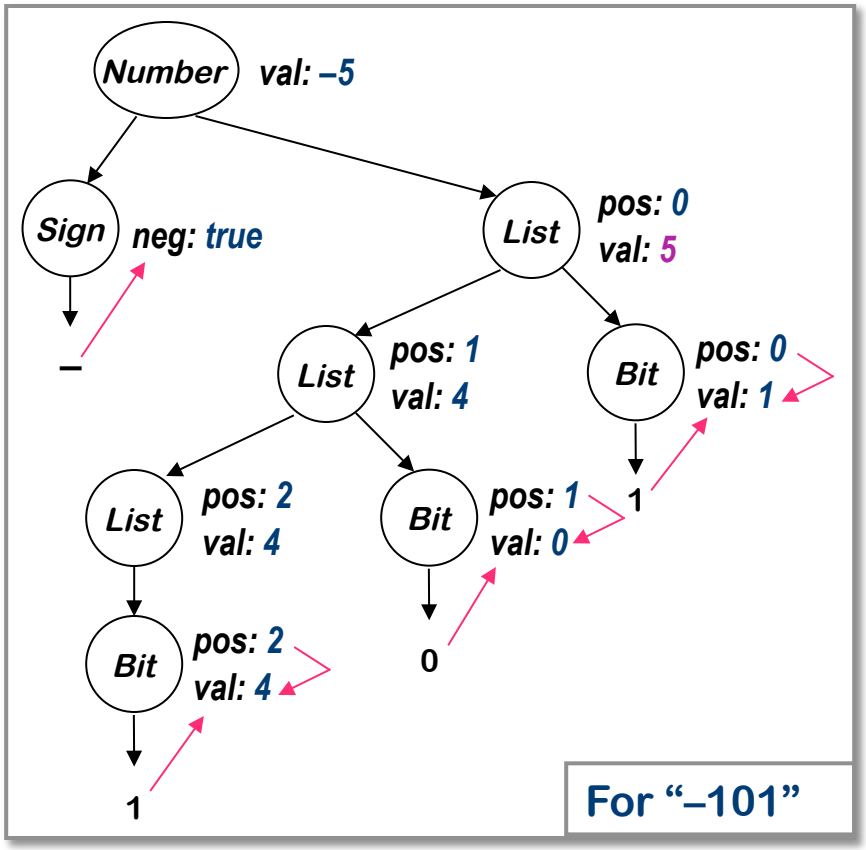
Attributed Syntax Tree

Back to the Example



Inherited Attributes

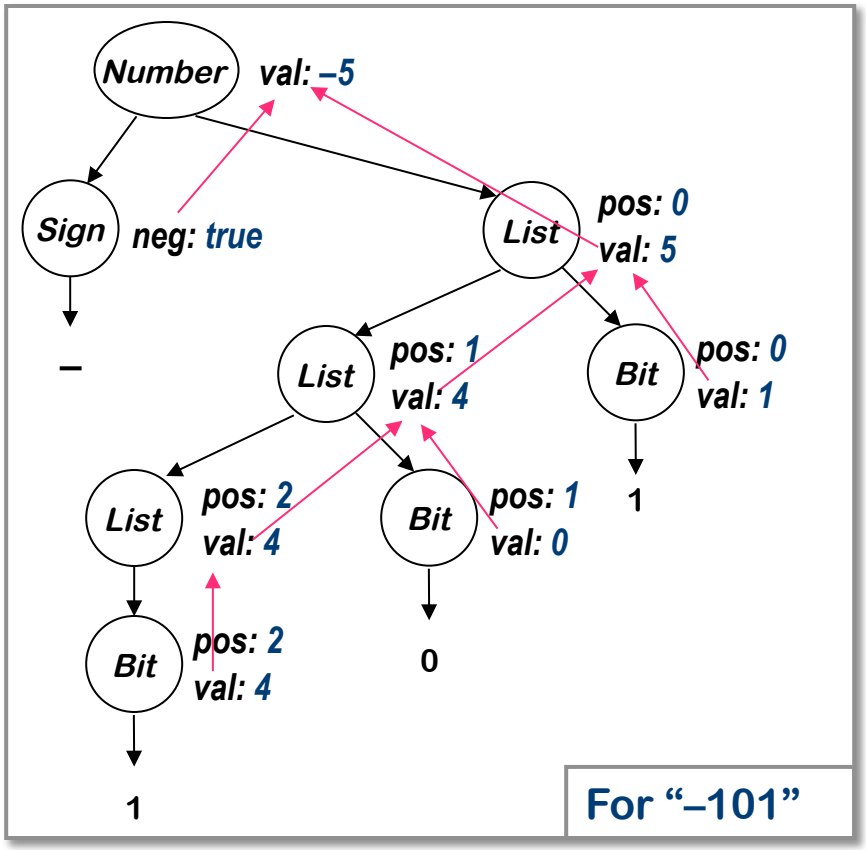
Back to the Example



Synthesized attributes

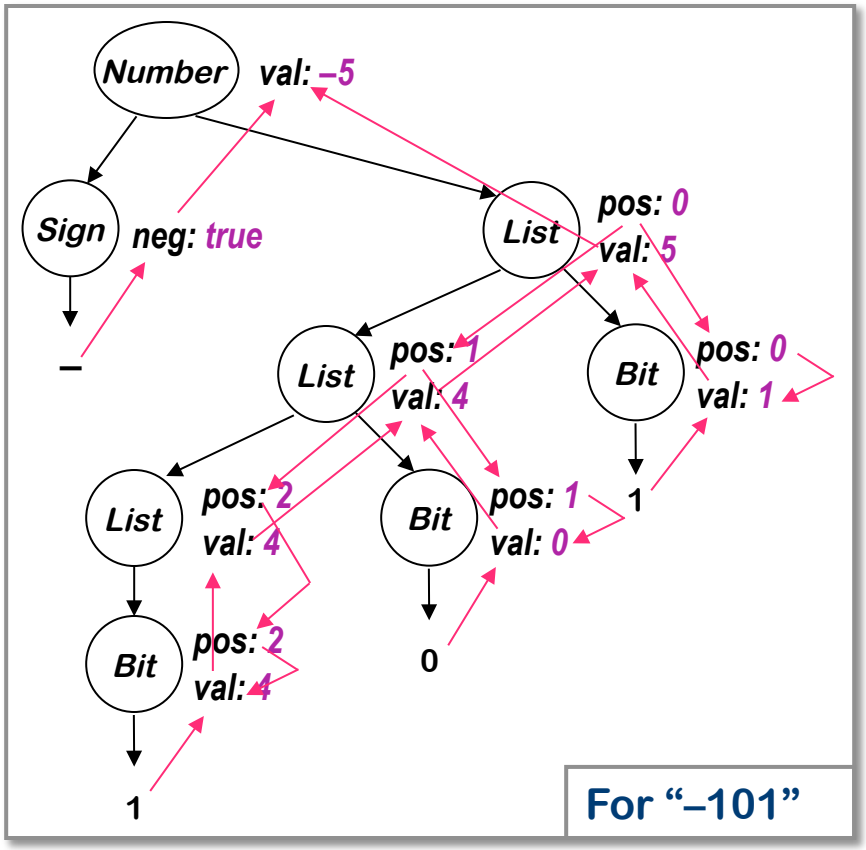
Val draws from children & the same node.

Back to the Example



More Synthesized attributes

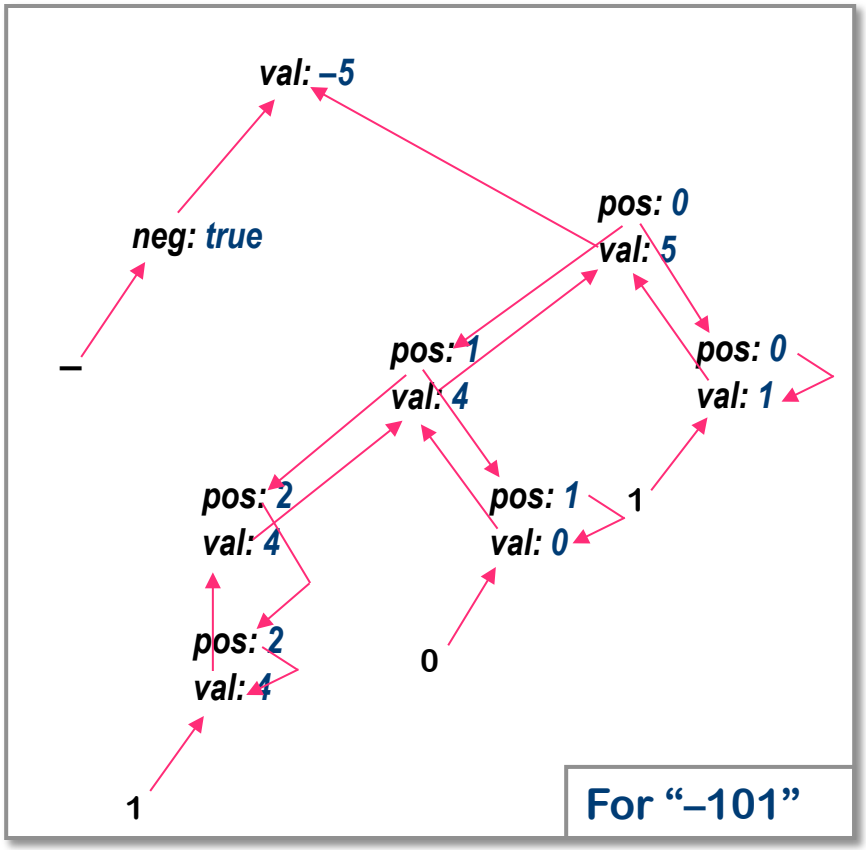
Back to the Example



If we show the computation …

& then peel away the parse tree …

Back to the Example



All that is left is the attribute dependence graph.

This succinctly represents the flow of values in the problem instance.

The dynamic methods sort this graph to find independent values, then work along graph edges.

The rule-based methods try to discover "good" orders by analyzing the rules.

The oblivious methods ignore the structure of this graph.

The dependence graph *must* be acyclic

Circularity

We can only evaluate acyclic instances

- General circularity testing is inherently exponential!
- We can prove that some grammars can only generate instances with acyclic dependence graphs
  - Largest such class is "strongly non-circular" grammars (*SNC*)
  - *SNC* grammars can be tested in polynomial time
  - Failing the *SNC* test is <u>not</u> conclusive

Many evaluation methods discover circularity dynamically

⇒ Bad property for a compiler to have

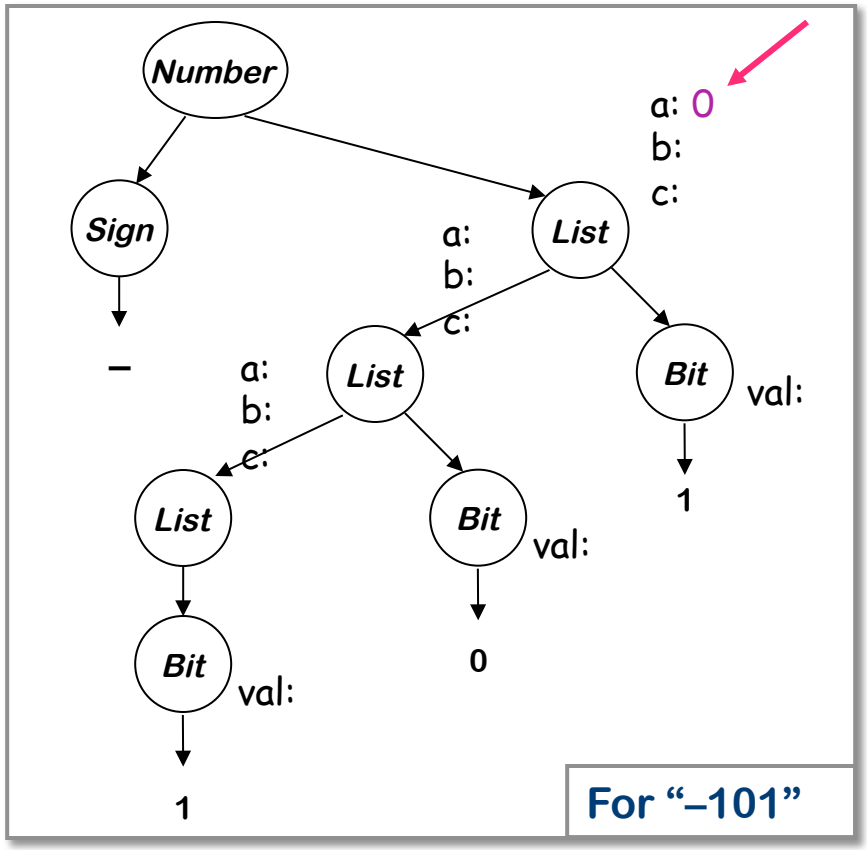
SNC comes from Kennedy & Warren, POPL 1976

A Circular Attribute Grammar

Productions			Attribution Rules
Number	$\rightarrow$	*List*	$List.a \leftarrow 0$
$List_0$	$\rightarrow$	$List_1$ *Bit*	$List_1.a \leftarrow List_0.a + 1$
			$List_0.b \leftarrow List_1.b$
			$\boxed{List_1.c \leftarrow List_1.b + Bit.val}$
	|	*Bit*	$\boxed{List_0.b \leftarrow List_0.a + List_0.c + Bit.val}$
Bit	$\rightarrow$	*0*	$Bit.val \leftarrow 0$
	|	*1*	$Bit.val \leftarrow 1$

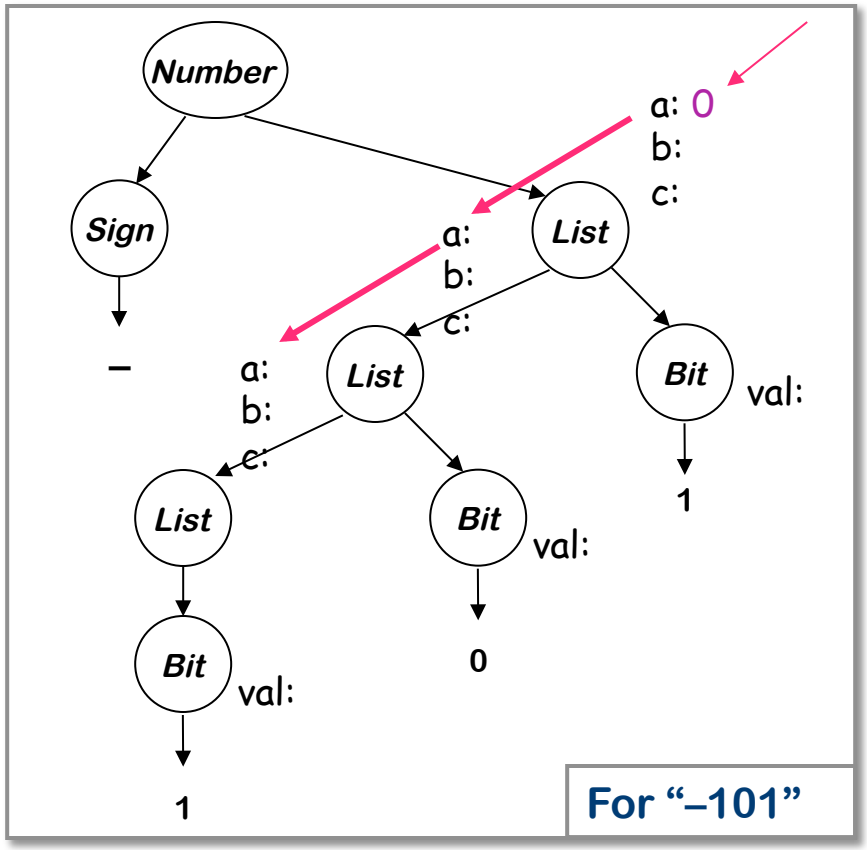
Remember, the circularity is in the attribution rules, not the underlying CFG

Circular Grammar Example



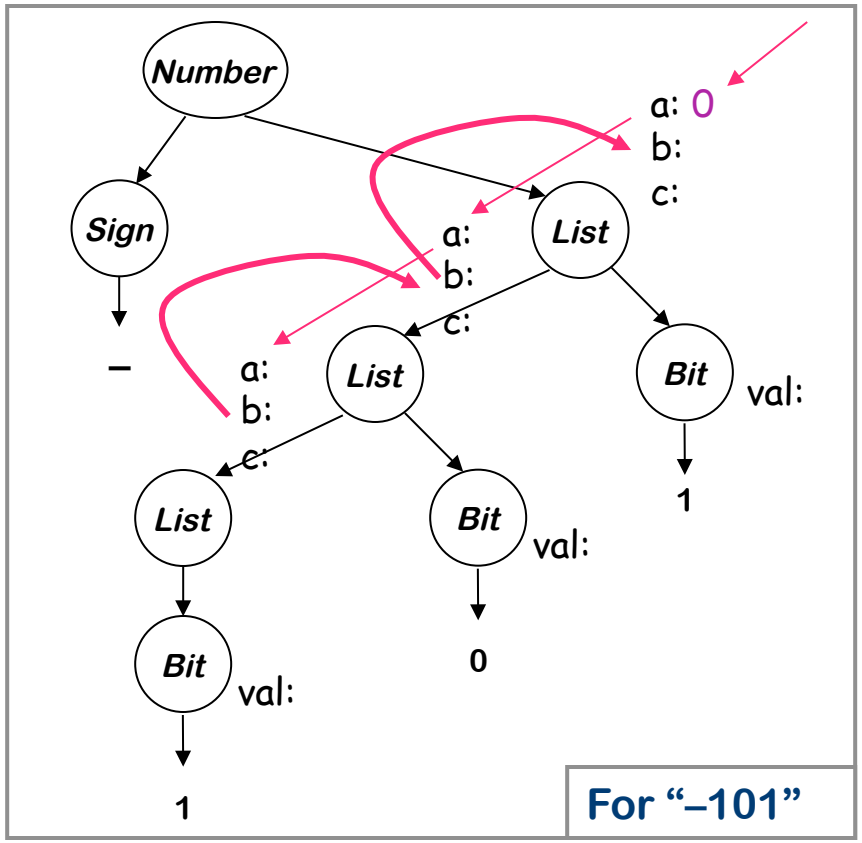
Productions			Attribution Rules
Number	→	*List*	$\boxed{List.a \leftarrow 0}$
$List_0$	→	$List_1$ *Bit*	$List_1.a \leftarrow List_0.a + 1$ $List_0.b \leftarrow List_1.b$ $List_1.c \leftarrow List_1.b + Bit.val$
	|	*Bit*	$List_0.b \leftarrow List_0.a + List_0.c + Bit.val$
Bit	→	*0*	$Bit.val \leftarrow 0$
	|	*1*	$Bit.val \leftarrow 1$

Circular Grammar Example



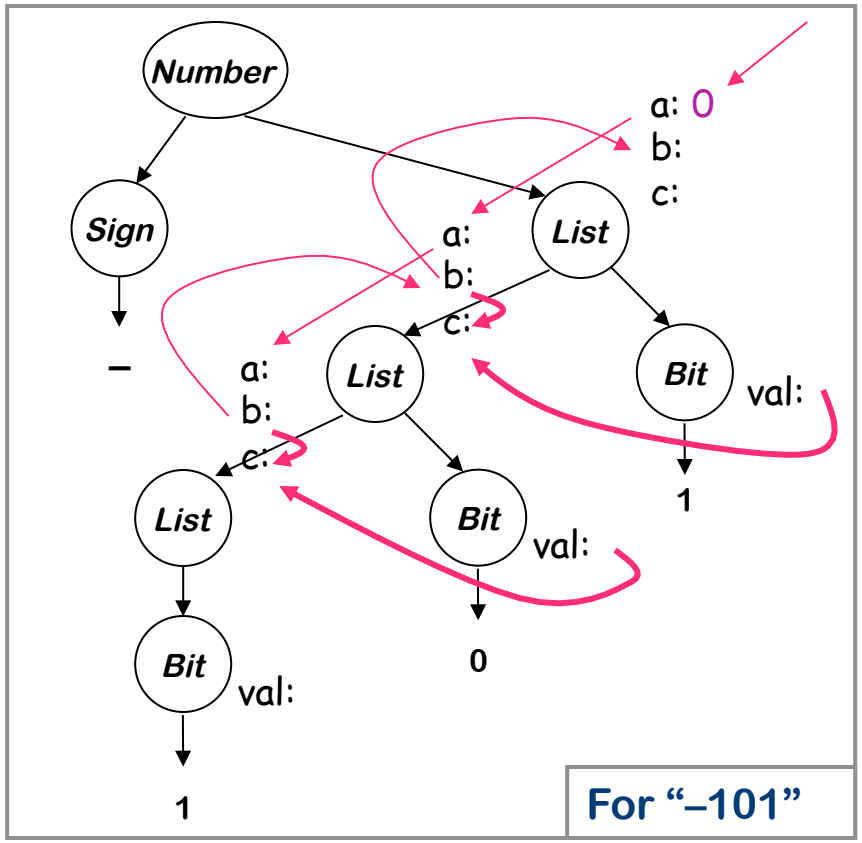
Productions			Attribution Rules
Number	→	*List*	$List.a \leftarrow 0$
$List_0$	→	$List_1$ *Bit*	$\boxed{List_1.a \leftarrow List_0.a + 1}$
			$List_0.b \leftarrow List_1.b$
			$List_1.c \leftarrow List_1.b + Bit.val$
	|	*Bit*	$List_0.b \leftarrow List_0.a + List_0.c + Bit.val$
Bit	→	*0*	$Bit.val \leftarrow 0$
	|	*1*	$Bit.val \leftarrow 1$

Circular Grammar Example



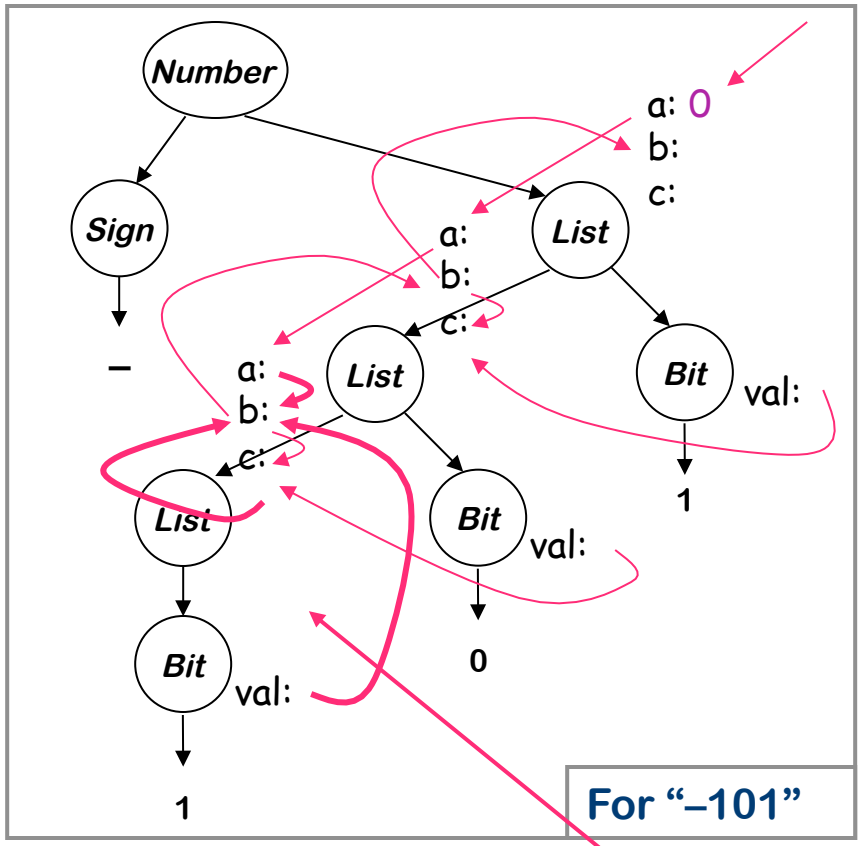
Productions			Attribution Rules
Number	→	*List*	$List.a \leftarrow 0$
$List_0$	→	$List_1$ *Bit*	$List_1.a \leftarrow List_0.a + 1$
			$\boxed{List_0.b \leftarrow List_1.b}$
			$List_1.c \leftarrow List_1.b + Bit.val$
	|	*Bit*	$List_0.b \leftarrow List_0.a + List_0.c + Bit.val$
Bit	→	*0*	$Bit.val \leftarrow 0$
	|	*1*	$Bit.val \leftarrow 1$

Circular Grammar Example



Productions			Attribution Rules
Number	→	*List*	$List.a \leftarrow 0$
$List_0$	→	$List_1$ *Bit*	$List_1.a \leftarrow List_0.a + 1$ $List_0.b \leftarrow List_1.b$ $List_1.c \leftarrow List_1.b + Bit.val$
	|	*Bit*	$List_0.b \leftarrow List_0.a + List_0.c + Bit.val$
Bit	→	*0*	$Bit.val \leftarrow 0$
	|	*1*	$Bit.val \leftarrow 1$

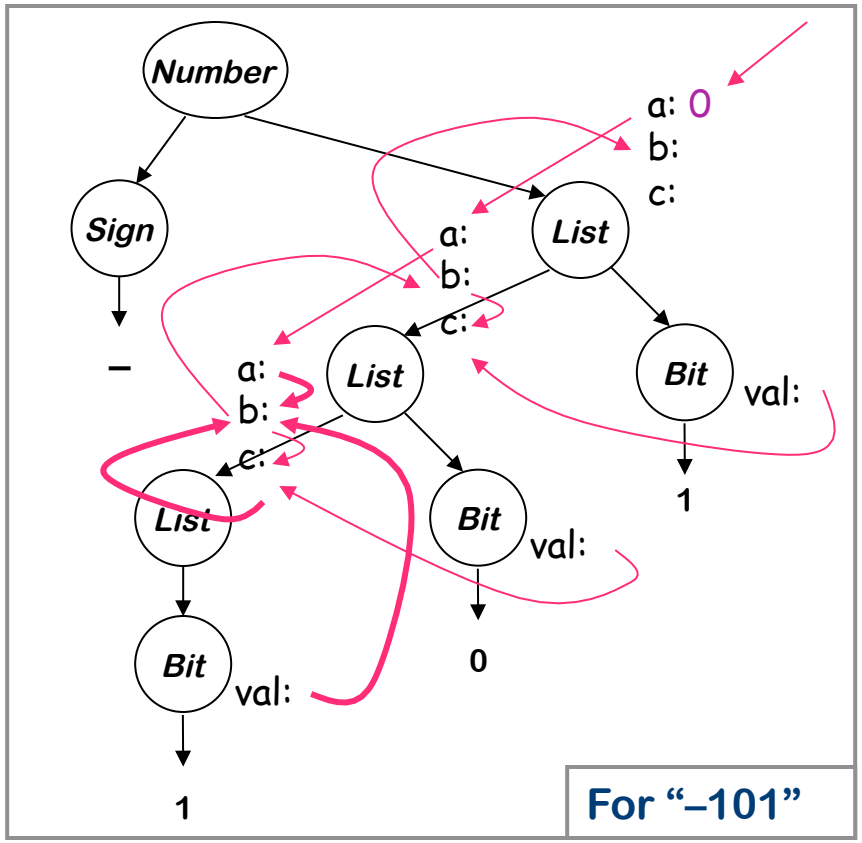
Circular Grammar Example



Productions			Attribution Rules
Number	→	*List*	$List.a \leftarrow 0$
$List_0$	→	$List_1$ *Bit*	$List_1.a \leftarrow List_0.a + 1$ $List_0.b \leftarrow List_1.b$ $List_1.c \leftarrow List_1.b + Bit.val$
	|	*Bit*	$List_0.b \leftarrow List_0.a + List_0.c + Bit.val$
Bit	→	*0*	$Bit.val \leftarrow 0$
	|	*1*	$Bit.val \leftarrow 1$

Here is the circularity …

Circular Grammar Example



Productions			Attribution Rules
Number	→	*List*	$List.a \leftarrow 0$
$List_0$	→	$List_1$	$List_1.a \leftarrow List_0.a + 1$
		Bit	$List_0.b \leftarrow List_1.b$
			$List_1.c \leftarrow List_1.b + Bit.val$
	|	*Bit*	$List_0.b \leftarrow List_0.a + List_0.c + Bit.val$
Bit	→	*0*	$Bit.val \leftarrow 0$
	|	*1*	$Bit.val \leftarrow 1$

Here is the circularity …

Circularity — The Point

- Circular grammars have indeterminate values
  - Algorithmic evaluators will fail
- Noncircular grammars evaluate to a unique set of values
- Circular grammar might give rise to noncircular instance
  - Probably shouldn't bet the compiler on it …

⇒Should (*undoubtedly*) use provably noncircular grammars

Remember, we are studying AGs to gain insight

- We should avoid circular, indeterminate computations
- If we stick to provably noncircular schemes, evaluation should be easier

An Extended Attribute Grammar Example

Grammar for a basic block *(§ 4.3.3)*

1	$Block_0$	→	$Block_1$ *Assign*
2		|	*Assign*
3	$Assign_0$	→	*Ident* = *Expr* ;
4	$Expr_0$	→	$Expr_1$ + *Term*
5		|	$Expr_1$ - *Term*
6		|	*Term*
7	$Term_0$	→	$Term_1$ * *Factor*
8		|	$Term_1$ / *Factor*
9		|	*Factor*
10	*Factor*	→	(*Expr*)
11		|	*Number*
12		|	*Ident*

Let's estimate cycle counts

- Each operation has a COST
- Add them, bottom up
- Assume a load per value
- Assume no reuse

Simple problem for an *AG*

Hey, that might be a practical application!

An Extended Example (*continued*)

1	$Block_0$	→	$Block_1$ *Assign*	$Block_0$.*cost* ← $Block_1$.*cost* + *Assign.cost*
2		|	*Assign*	$Block_0$.*cost* ← *Assign.cost*
3	$Assign_0$	→	*Ident* = *Expr* ;	*Assign.cost* ← COST(store) + *Expr.cost*
4	$Expr_0$	→	$Expr_1$ + *Term*	$Expr_0$.*cost* ← $Expr_1$.*cost* + COST(add) + *Term.cost*
5		|	$Expr_1$ - *Term*	$Expr_0$.*cost* ← $Expr_1$.*cost* + COST(sub) + *Term.cost*
6		|	*Term*	$Expr_0$.*cost* ← *Term.cost*
7	$Term_0$	→	$Term_1$ * *Factor*	$Term_0$.*cost* ← $Term_1$.*cost* + COST(mult) + *Factor.cost*
8		|	$Term_1$ / *Factor*	$Term_0$.*cost* ← $Term_1$.*cost* + COST(div) + *Factor.cost*
9		|	*Factor*	$Term_0$.*cost* ← *Factor.cost*
10	*Factor*	→	(*Expr*)	*Factor.cost* ← *Expr.cost*
11		|	*Number*	*Factor.cost* ← COST(loadI)
12		|	*Ident*	*Factor.cost* ← COST(load)

These are all synthesized attributes !

Values flow from *rhs* to *lhs* in prod'ns

An Extended Example (*continued*)

Properties of the example grammar

- All attributes are synthesized ⇒ S-attributed grammar
- Rules can be evaluated bottom-up in a single pass
  - Good fit to bottom-up, shift/reduce parser
- Easily understood solution
- Seems to fit the problem well

What about an improvement?

- Values are loaded only once per block (not at each use)
- Need to track which values have been already loaded

A Better Execution Model

Adding load tracking

- Need sets *Before* and *After* for each production
- Must be initialized, updated, and passed around the tree

10	*Factor* → (*Expr*)	*Factor.cost* ← *Expr.cost* *Expr.before* ← *Factor.before* *Factor.after* ← *Expr.after*
11	| *Number*	*Factor.cost* ← COST(loadI) *Factor.after* ← *Factor.before*
12	| *Ident*	*If (Ident.name* ∉ *Factor.before)* *then* *Factor.cost* ← COST(load) *Factor.after* ← *Factor.before* ∪ { *Ident.name* } *else* *Factor.cost* ← 0 *Factor.after* ← *Factor.before*

This version is much more complex

A Better Execution Model

- Load tracking adds complexity
- But, most of it is in the "copy rules"
- Every production needs rules to copy *Before* & *After*

A sample production

4	$Expr_0 \rightarrow Expr_1 + Term$	$Expr_0.cost \leftarrow Expr_1.cost + COST(add) + Term.cost$ $Expr_1.before \leftarrow Expr_0.before$ $Term.before \leftarrow Expr_1.before$ $Expr_0.after \leftarrow Term.after$

These copy rules multiply rapidly

Each creates an instance of the set

Lots of work, lots of space, lots of rules to write

An Even Better Model

What about accounting for finite register sets?

- *Before* & *After* must be of limited size
- Adds complexity to *Factor→Identifier*
- Requires more complex initialization

Jump from tracking loads to tracking registers is small

- Copy rules are already in place
- Some local code to perform the allocation

And Its Extensions

Tracking loads

- Introduced *Before* and *After* sets to record loads
- Added ≥ 2 copy rules per production
  - Serialized evaluation into execution order
- Made the whole attribute grammar large & cumbersome

Finite register set

- Complicated one production (*Factor* → Identifier)
- Needed a little fancier initialization
- Changes were quite limited

Why is one change hard and the other easy?

The Moral of the Story

- Non-local computation needed lots of supporting rules
- Complex local computation was relatively easy

The Problems

- Copy rules increase cognitive overhead
- Copy rules increase space requirements
  - Need copies of attributes
  - Can use pointers, for even more cognitive overhead
- Result is an attributed tree *(somewhat subtle points)*
  - Must build the parse tree
  - Either search tree for answers or copy them to the root

A good rule of thumb is that the compiler touches all the space it allocates, usually multiple times